

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Amandha Citra Mustika NIM. 2410817320004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Amandha Citra Mustika
NIM : 2410817320004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL.....	5
SOAL 1	6
A. Source Code	7
B. Output Program.....	34
C. Pembahasan.....	38
SOAL 2	48
A. Pembahasan.....	48
TAUTAN GIT	49

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Test Debugging Soal 1 XML	34
Gambar 2. Screenshot Hasil Test Debugging Soal 1 XML	34
Gambar 3. Screenshot Hasil Test Debugging Soal 1 XML	35
Gambar 4. Screenshot Hasil Test Debugging Soal 1 XML	35
Gambar 5. Screenshot Hasil Test Debugging Soal 1 Compose.....	36
Gambar 6. Screenshot Hasil Test Debugging Soal 1 Compose.....	36
Gambar 7. Screenshot Hasil Test Debugging Soal 1 Compose.....	37
Gambar 8. Screenshot Hasil Test Debugging Soal 1 Compose.....	37

DAFTAR TABEL

Table 1. Source Code MainActivity.kt	7
Table 2. Source Code ComicAdapter.kt	7
Table 3. Source Code DetailFargment.kt.....	8
Table 4. Source Code HighlightAdapter.kt.....	9
Table 5. Source Code ListFragment.kt	10
Table 6. Source Code LanguageFragment.kt.....	12
Table 7. Source Code ComicViewModel.kt	13
Table 8. Source Code ComicViewModelFactory.kt.....	13
Table 9. Source Code ComicDataSource.kt	14
Table 10. Source Code Comic.kt	15
Table 11. Source Code activity_main.xml.....	15
Table 12. Source Code fragment_comic_detail.xml.....	15
Table 13. Source Code fragment_comic_detail.xml.....	17
Table 14. Source Code fragment_comic_list.xml.....	18
Table 15. Source Code fragment_language.xml.....	19
Table 16. Source Code item_comic.xml.....	20
Table 17. Source Code item_comic_highlight.xml	21
Table 18. Source Code ComicDataSource.kt	23
Table 19. Source Code ComicDetailScreen.kt	23
Table 20. Source Code ComicListScreen.kt	25
Table 21. Source Code ComicViewModel.kt	28
Table 22. Source Code ComicViewModelFactory.kt.....	29
Table 23. Source Code LanguageScreen.kt	30
Table 24. Source Code MainActivity.kt	31
Table 25. Source Code MyApp.kt	32
Table 26. Source Code Comic.kt	32
Table 27. Source Code ComicDataSource.kt	33

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - d. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.

A. Source Code

XML

MainActivity.kt

Table 1. Source Code MainActivity.kt

1	package com.example.mycomicappxmlm4
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import com.example.mycomicappxmlm4.databinding.ActivityMainBinding
6	import timber.log.Timber
7	
8	class MainActivity : AppCompatActivity() {
9	
10	private lateinit var binding: ActivityMainBinding
11	
12	override fun onCreate(savedInstanceState: Bundle?) {
13	super.onCreate(savedInstanceState)
14	binding = ActivityMainBinding.inflate(layoutInflater)
15	setContentView(binding.root)
16	}
17	}

ComicAdapter.kt

Table 2. Source Code ComicAdapter.kt

1	package com.example.mycomicappxmlm4.ui
2	
3	import android.content.Intent
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.core.net.toUri
7	import androidx.recyclerview.widget.RecyclerView
8	import com.example.mycomicappxmlm4.R
9	import com.example.mycomicappxmlm4.data.Comic
10	import com.example.mycomicappxmlm4.databinding.ItemComicBinding
11	
12	class ComicAdapter(
13	private val comics: List<Comic>,
14	private val onDetailClick: (Int) -> Unit,
15	private val onExplicitIntentClick: (Comic) -> Unit
16	) : RecyclerView.Adapter<ComicAdapter.ViewHolder>() {
17	
18	inner class ViewHolder(val binding: ItemComicBinding) :
19	RecyclerView.ViewHolder(binding.root)
20	
21	override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder
22	{
23	val binding = ItemComicBinding.inflate(
24	LayoutInflater.from(parent.context), parent, false
25	)
26	return ViewHolder(binding)
27	}

28	
29	override fun onBindViewHolder(holder: ViewHolder, position: Int) {
30	val comic = comics[position]
31	val context = holder.itemView.context
32	with(holder.binding) {
33	imgComicCover.setImageResource(comic.imageResId)
34	tvComicTitle.text = comic.title
35	tvComicYear.text = comic.year
36	tvComicGenre.text = context.getString(R.string.label_genre) +
	comic.genre
37	tvComicAuthor.text = comic.author
38	
39	btnRead.setOnClickListener {
40	onExplicitIntentClick(comic)
41	val intent = Intent(Intent.ACTION_VIEW, comic.webtoonUrl.toUri())
42	context.startActivity(intent)
43	}
44	
45	btnDetail.setOnClickListener {
46	onDetailClick(comic.id)
47	}
48	}
49	}
50	
51	override fun getItemCount() = comics.size
	}

DetailFragment.kt

Table 3. Source Code DetailFargment.kt

1	package com.example.mycomicappxmlm4.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.viewModels
9	import androidx.lifecycle.LifecycleScope
10	import com.example.mycomicappxmlm4.R
11	import com.example.mycomicappxmlm4.databinding.FragmentComicDetailBinding
12	import com.example.mycomicappxmlm4.viewmodel.ComicViewModel
13	import com.example.mycomicappxmlm4.viewmodel.ComicViewModelFactory
14	import kotlinx.coroutines.launch
15	import timber.log.Timber
16	
17	class ComicDetailFragment : Fragment() {
18	
19	private var _binding: FragmentComicDetailBinding? = null
20	private val binding get() = _binding!!
21	
22	private val viewModel: ComicViewModel by viewModels {
23	ComicViewModelFactory("MyComicAppXMLM4", requireContext())
24	}
25	
26	override fun onCreateView(
27	inflater: LayoutInflater,
28	container: ViewGroup?,


```

29         savedInstanceState: Bundle?
30     ): View {
31         _binding = FragmentComicDetailBinding.inflate(inflater, container, false)
32         return binding.root
33     }
34
35     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
36         super.onCreateView(view, savedInstanceState)
37
38         val comicId = arguments?.getInt("comicId") ?: return
39
40         viewLifecycleOwner.lifecycleScope.launch {
41             viewModel.comicList.collect { comics ->
42                 val comic = comics.find { it.id == comicId }
43                 comic?.let {
44                     Timber.d("Data comic di halaman Detail: ${it.title}")
45                     viewModel.selectComic(it)
46                     binding.imgDetailCover.setImageResource(it.imageResId)
47                     binding.tvDetailTitle.text = it.title
48                     binding.tvDetailGenre.text = getString(R.string.label_genre)
49                     + it.genre
50                     binding.tvDetailAuthor.text =
51                     getString(R.string.label_author) + it.author
52                     binding.tvDetailYear.text = getString(R.string.label_year) +
53                     it.year
54                     binding.tvDetailSynopsis.text = it.synopsis
55                 }
56             }
57         }
58
59         override fun onDestroyView() {
60             super.onDestroyView()
61             _binding = null
62         }
63     }

```

HighlightAdapter.kt

Table 4. Source Code HighlightAdapter.kt

```

1 package com.example.mycomicappxmlm4.ui
2
3 import android.view.LayoutInflater
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.RecyclerView
6 import com.example.mycomicappxmlm4.data.Comic
7 import com.example.mycomicappxmlm4.databinding.ItemComicHighlightBinding
8
9 class ComicHighlightAdapter(
10     private val comics: List<Comic>,
11     private val onDetailClick: (Int) -> Unit
12 ) : RecyclerView.Adapter<ComicHighlightAdapter.ViewHolder>() {
13
14     inner class ViewHolder(val binding: ItemComicHighlightBinding) :
15         RecyclerView.ViewHolder(binding.root)
16
17     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder
18     {

```

18	val binding = ItemComicHighlightBinding.inflate(
19	LayoutInflater.from(parent.context), parent, false
20	)
21	return ViewHolder(binding)
22	}
23	
24	override fun onBindViewHolder(holder: ViewHolder, position: Int) {
25	val comic = comics[position]
26	with(holder.binding) {
27	imgHighlightCover.setImageResource(comic.imageResId)
28	tvHighlightTitle.text = comic.title
29	tvHighlightGenre.text = comic.genre
30	
31	btnHighlightDetail.setOnClickListener {
32	onDetailClick(comic.id)
33	}
34	}
35	}
36	
37	override fun getItemCount() = comics.size
38	}

ListFragment.kt

Table 5. Source Code ListFragment.kt

1	package com.example.mycomicappxmlm4.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.viewModels
9	import androidx.lifecycle.lifecycleScope
10	import androidx.navigation.fragment.findNavController
11	import androidx.recyclerview.widget.LinearLayoutManager
12	import com.example.mycomicappxmlm4.R
13	import com.example.mycomicappxmlm4.databinding.FragmentComicListBinding
14	import com.example.mycomicappxmlm4.viewmodel.ComicViewModel
15	import com.example.mycomicappxmlm4.viewmodel.ComicViewModelFactory
16	import kotlinx.coroutines.launch
17	import timber.log.Timber
18	
19	class ComicListFragment : Fragment() {
20	
21	private var _binding: FragmentComicListBinding? = null
22	private val binding get() = _binding!!
23	
24	private val viewModel: ComicViewModel by viewModels {
25	ComicViewModelFactory("MyComicAppXMLM4", requireContext())
26	}
27	
28	private lateinit var comicAdapter: ComicAdapter
29	private lateinit var highlightAdapter: ComicHighlightAdapter
30	
31	override fun onCreateView(
32	inflater: LayoutInflater,
33	container: ViewGroup?,

```

34         savedInstanceState: Bundle?
35     ): View {
36         _binding = FragmentComicListBinding.inflate(inflater, container, false)
37         return binding.root
38     }
39
40     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
41         super.onViewCreated(view, savedInstanceState)
42
43         binding.rvHighlight.layoutManager = LinearLayoutManager(
44             requireContext(), LinearLayoutManager.HORIZONTAL, false
45         )
46         binding.rvComicList.layoutManager = LinearLayoutManager(requireContext())
47
48         viewLifecycleOwner.lifecycleScope.launch {
49             viewModel.comicList.collect { comics ->
50                 highlightAdapter = ComicHighlightAdapter(comics) { comicId ->
51                     navigateToDetail(comicId)
52                 }
53                 binding.rvHighlight.adapter = highlightAdapter
54
55                 comicAdapter = ComicAdapter(
56                     comics = comics,
57                     onDetailClick = { comicId ->
58                         val comic = comics.find { it.id == comicId }
59                         comic?.let {
60                             Timber.d("Tombol Detail ditekan: ${it.title}")
61                             viewModel.onDetailClicked(it)
62                         }
63                         navigateToDetail(comicId)
64                     },
65                     onExplicitIntentClick = { comic ->
66                         viewModel.onExplicitIntentClicked(comic)
67                     }
68                 )
69                 binding.rvComicList.adapter = comicAdapter
70             }
71         }
72
73         binding.btnLanguage.setOnClickListener {
74             findNavController().navigate(R.id.action_comicListFragment_to_languageFragment)
75         }
76     }
77
78     private fun navigateToDetail(comicId: Int) {
79         val bundle = Bundle().apply { putInt("comicId", comicId) }
80         findNavController().navigate(
81             R.id.action_comicListFragment_to_comicDetailFragment, bundle
82         )
83     }
84
85     override fun onDestroyView() {
86         super.onDestroyView()
87         _binding = null
88     }
89 }

```

LanguageFragment.kt

Table 6. Source Code LanguageFragment.kt

1	package com.example.mycomicappxmlm4.ui
2	
3	import android.content.res.Configuration
4	import android.os.Bundle
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment
9	import androidx.navigation.fragment.findNavController
10	import com.example.mycomicappxmlm4.databinding.FragmentLanguageBinding
11	import java.util.Locale
12	
13	class LanguageFragment : Fragment() {
14	
15	private var _binding: FragmentLanguageBinding? = null
16	private val binding get() = _binding!!
17	
18	override fun onCreateView(
19	inflater: LayoutInflater,
20	container: ViewGroup?,
21	savedInstanceState: Bundle?
22	): View {
23	_binding = FragmentLanguageBinding.inflate(inflater, container, false)
24	return binding.root
25	}
26	
27	override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
28	super.onViewCreated(view, savedInstanceState)
29	
30	binding.btnEnglish.setOnClickListener {
31	setLocale("en")
32	}
33	
34	binding.btnIndonesian.setOnClickListener {
35	setLocale("in")
36	}
37	}
38	
39	private fun setLocale(languageCode: String) {
40	val locale = Locale(languageCode)
41	Locale.setDefault(locale)
42	val config = Configuration(resources.configuration)
43	config.setLocale(locale)
44	requireActivity().resources.updateConfiguration(config,
45	requireActivity().resources.displayMetrics)
46	requireActivity().recreate()
47	}
48	
49	override fun onDestroyView() {
50	super.onDestroyView()
51	_binding = null
52	}

ComicViewModel.kt

Table 7. Source Code ComicViewModel.kt

1	package com.example.mycomicappxmlm4.viewmodel
2	
3	import android.content.Context
4	import androidx.lifecycle.ViewModel
5	import com.example.mycomicappxmlm4.data.Comic
6	import com.example.mycomicappxmlm4.data.ComicDataSource
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import timber.log.Timber
10	
11	class ComicViewModel(private val appName: String, private val context: Context) : ViewModel() {
12	
13	private val _comicList = MutableStateFlow<List<Comic>>(emptyList())
14	val comicList: StateFlow<List<Comic>> = _comicList
15	
16	private val _selectedComic = MutableStateFlow<Comic?>(null)
17	val selectedComic: StateFlow<Comic?> = _selectedComic
18	
19	init {
20	loadComics()
21	}
22	
23	private fun loadComics() {
24	val comics = ComicDataSource.getComicList(context)
25	_comicList.value = comics
26	Timber.d("[\${appName}] \${comics.size} item comic berhasil dimuat ke dalam list")
27	}
28	
29	fun selectComic(comic: Comic) {
30	_selectedComic.value = comic
31	Timber.d("[\${appName}] Comic dipilih: \${comic.title}")
32	}
33	
34	fun onDetailClicked(comic: Comic) {
35	Timber.d("[\${appName}] Tombol Detail ditekan untuk: \${comic.title}")
36	selectComic(comic)
37	}
38	
39	fun onExplicitIntentClicked(comic: Comic) {
40	Timber.d("[\${appName}] Tombol Explicit Intent ditekan untuk: \${comic.title}")
41	}
42	}
	}

ComicViewModelFactory.kt

Table 8. Source Code ComicViewModelFactory.kt

1	package com.example.mycomicappxmlm4.viewmodel
2	
3	import android.content.Context

```

4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6
7 class ComicViewModelFactory(
8     private val appName: String,
9     private val context: Context
10 ) : ViewModelProvider.Factory {
11
12     override fun <T : ViewModel> create(modelClass: Class<T>): T {
13         if (modelClass.isAssignableFrom(ComicViewModel::class.java)) {
14             @Suppress("UNCHECKED_CAST")
15             return ComicViewModel(appName, context) as T
16         }
17         throw IllegalArgumentException("Unknown ViewModel class")
18     }
19 }

```

ComicDataSource.kt

Table 9. Source Code ComicDataSource.kt

```

1 package com.example.mycomicappxmlm4.data
2
3 import android.content.Context
4 import com.example.mycomicappxmlm4.R
5
6 object ComicDataSource {
7     fun getComicList(context: Context) = listOf(
8         Comic(1, "Like Mother Like Daughter", "Thriller", "Carnby Kim", "2023",
9             context.getString(R.string.synopsis_like_mother),
10             R.drawable.lmld,
11             "https://www.webtoons.com/id/thriller/like-mother-like-
12 daughter/list?title_no=3622"),
13         Comic(2, "The Real Lesson", "Action", "Hyun", "2022",
14             context.getString(R.string.synopsis_real_lesson),
15             R.drawable.thereallesson,
16             "https://www.webtoons.com/id/action/the-real-
17 lesson/list?title_no=2423"),
18         Comic(3, "Wee!", "Slice of Life", "Soonkki", "2023",
19             context.getString(R.string.synopsis_wee),
20             R.drawable.wee,
21             "https://www.webtoons.com/id/slice-of-life/wee/list?title_no=3085"),
22         Comic(4, "The Villainess Reverses the Hourglass", "Romantic Fantasy",
23             "Sansobee", "2021",
24             context.getString(R.string.synopsis_villainess),
25             R.drawable.villainess,
26             "https://www.webtoons.com/id/romantic-fantasy/the-villainess-
27 reverses-hourglass/list?title_no=9517"),
28         Comic(5, "The Fantasie of a Stepmother", "Romantic Fantasy", "Rukyung",
29             "2020",
30             context.getString(R.string.synopsis_stepmother),
31             R.drawable.stepmother,
32             "https://www.webtoons.com/id/romantic-fantasy/the-fantasie-of-a-
33 stepmother/list?title_no=9773")
34     )
35 }

```

Comic.kt

Table 10. Source Code Comic.kt

1	package com.example.mycomicappxmlm4.data
2	
3	data class Comic(
4	val id: Int,
5	val title: String,
6	val genre: String,
7	val author: String,
8	val year: String,
9	val synopsis: String,
10	val imageResId: Int,
11	val webtoonUrl: String
12	)

activity_main.xml

Table 11. Source Code activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent">
7	
8	<androidx.fragment.app.FragmentContainerView
9	android:id="@+id/nav_host_fragment"
10	android:name="androidx.navigation.fragment.NavHostFragment"
11	android:layout_width="match_parent"
12	android:layout_height="match_parent"
13	app:defaultNavHost="true"
14	app:navGraph="@navigation/nav_graph"
15	app:layout_constraintTop_toTopOf="parent"
16	app:layout_constraintBottom_toBottomOf="parent"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintEnd_toEndOf="parent"/>
19	
20	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_comic_detail.xml

Table 12. Source Code fragment_comic_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent">
6	
7	<LinearLayout
8	android:layout_width="match_parent"
9	android:layout_height="wrap_content"

```

10     android:orientation="vertical">
11
12     <ImageView
13         android:id="@+id/imgDetailCover"
14         android:layout_width="match_parent"
15         android:layout_height="350dp"
16         android:scaleType="centerCrop"
17         android:contentDescription="@string/cd_comic_cover"/>
18
19     <LinearLayout
20         android:layout_width="match_parent"
21         android:layout_height="wrap_content"
22         android:orientation="vertical"
23         android:padding="16dp">
24
25         <TextView
26             android:id="@+id/tvDetailTitle"
27             android:layout_width="match_parent"
28             android:layout_height="wrap_content"
29             android:textSize="24sp"
30             android:textStyle="bold"
31             android:layout_marginBottom="4dp"/>
32
33         <TextView
34             android:id="@+id/tvDetailGenre"
35             android:layout_width="match_parent"
36             android:layout_height="wrap_content"
37             android:textSize="14sp"
38             android:layout_marginBottom="4dp"/>
39
40         <TextView
41             android:id="@+id/tvDetailAuthor"
42             android:layout_width="match_parent"
43             android:layout_height="wrap_content"
44             android:textSize="14sp"
45             android:layout_marginBottom="4dp"/>
46
47         <TextView
48             android:id="@+id/tvDetailYear"
49             android:layout_width="match_parent"
50             android:layout_height="wrap_content"
51             android:textSize="14sp"
52             android:layout_marginBottom="12dp"/>
53
54         <TextView
55             android:id="@+id/tvDetailSynopsisLabel"
56             android:layout_width="match_parent"
57             android:layout_height="wrap_content"
58             android:text="@string/label_synopsis"
59             android:textSize="16sp"
60             android:textStyle="bold"/>
61
62         <TextView
63             android:id="@+id/tvDetailSynopsis"
64             android:layout_width="match_parent"
65             android:layout_height="wrap_content"
66             android:textSize="14sp"/>
67
68     </LinearLayout>
69 </LinearLayout>
70 </ScrollView>

```


Fragment_comic_detail.xml

Table 13. Source Code fragment_comic_detail.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <ScrollView
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent">
6
7      <LinearLayout
8          android:layout_width="match_parent"
9          android:layout_height="wrap_content"
10         android:orientation="vertical">
11
12         <ImageView
13             android:id="@+id/imgDetailCover"
14             android:layout_width="match_parent"
15             android:layout_height="350dp"
16             android:scaleType="centerCrop"
17             android:contentDescription="@string/cd_comic_cover"/>
18
19         <LinearLayout
20             android:layout_width="match_parent"
21             android:layout_height="wrap_content"
22             android:orientation="vertical"
23             android:padding="16dp">
24
25             <TextView
26                 android:id="@+id/tvDetailTitle"
27                 android:layout_width="match_parent"
28                 android:layout_height="wrap_content"
29                 android:textSize="24sp"
30                 android:textStyle="bold"
31                 android:layout_marginBottom="4dp"/>
32
33             <TextView
34                 android:id="@+id/tvDetailGenre"
35                 android:layout_width="match_parent"
36                 android:layout_height="wrap_content"
37                 android:textSize="14sp"
38                 android:layout_marginBottom="4dp"/>
39
40             <TextView
41                 android:id="@+id/tvDetailAuthor"
42                 android:layout_width="match_parent"
43                 android:layout_height="wrap_content"
44                 android:textSize="14sp"
45                 android:layout_marginBottom="4dp"/>
46
47             <TextView
48                 android:id="@+id/tvDetailYear"
49                 android:layout_width="match_parent"
50                 android:layout_height="wrap_content"
51                 android:textSize="14sp"
```

52	android:layout_marginBottom="12dp"/>
53	
54	<TextView
55	android:id="@+id/tvDetailSynopsisLabel"
56	android:layout_width="match_parent"
57	android:layout_height="wrap_content"
58	android:text="@string/label_synopsis"
59	android:textSize="16sp"
60	android:textStyle="bold"/>
61	
62	<TextView
63	android:id="@+id/tvDetailSynopsis"
64	android:layout_width="match_parent"
65	android:layout_height="wrap_content"
66	android:textSize="14sp"/>
67	
68	</LinearLayout>
69	</LinearLayout>
70	</ScrollView>

fragment_comic_list.xml

Table 14. Source Code fragment_comic_list.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.core.widget.NestedScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent">
6	
7	<LinearLayout
8	android:layout_width="match_parent"
9	android:layout_height="wrap_content"
10	android:orientation="vertical"
11	android:padding="16dp">
12	
13	<LinearLayout
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:orientation="horizontal"
17	android:gravity="center_vertical"
18	android:layout_marginBottom="8dp">
19	
20	<TextView
21	android:layout_width="0dp"
22	android:layout_height="wrap_content"
23	android:layout_weight="1"
24	android:text="@string/title_highlight"
25	android:textSize="20sp"
26	android:textStyle="bold"/>
27	
28	<com.google.android.material.button.MaterialButton
29	android:id="@+id/btnLanguage"
30	style="@style/Widget.Material3.Button.IconButton"
31	android:layout_width="wrap_content"
32	android:layout_height="wrap_content"
33	app:icon="@android:drawable/ic_menu_mylocation"

```

34         xmlns:app="http://schemas.android.com/apk/res-auto"/>
35
36     </LinearLayout>
37
38     <androidx.recyclerview.widget.RecyclerView
39         android:id="@+id/rvHighlight"
40         android:layout_width="match_parent"
41         android:layout_height="wrap_content"
42         android:orientation="horizontal"
43         android:paddingBottom="24dp"
44         android:clipToPadding="false"/>
45
46     <TextView
47         android:layout_width="wrap_content"
48         android:layout_height="wrap_content"
49         android:text="@string/title_all_comics"
50         android:textSize="20sp"
51         android:textStyle="bold"
52         android:layout_marginBottom="8dp"/>
53
54     <androidx.recyclerview.widget.RecyclerView
55         android:id="@+id/rvComicList"
56         android:layout_width="match_parent"
57         android:layout_height="wrap_content"
58         android:nestedScrollingEnabled="false"
59         android:paddingTop="12dp"/>
60
61 </LinearLayout>
62
63 </androidx.core.widget.NestedScrollView>

```

fragment_language.xml

Table 15. Source Code fragment_language.xml

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical"
7      android:gravity="center"
8      android:padding="32dp">
9
10     <TextView
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="@string/title_language"
14         android:textSize="22sp"
15         android:textStyle="bold"
16         android:layout_marginBottom="32dp"/>
17
18     <Button
19         android:id="@+id/btnEnglish"
20         android:layout_width="match_parent"
21         android:layout_height="wrap_content"
22         android:text="@string/btn_english"
23         android:layout_marginBottom="8dp"/>
24
25     <Button

```

26	android:id="@+id/btnIndonesian"
27	android:layout_width="match_parent"
28	android:layout_height="wrap_content"
29	android:text="@string/btn_indonesian"/>
30	
31	</LinearLayout>

item_comic.xml

Table 16. Source Code item_comic.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_marginBottom="16dp"
8	app:cardCornerRadius="16dp"
9	app:cardElevation="4dp">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="horizontal"
15	android:padding="12dp">
16	
17	<com.google.android.material.imageview.ShapeableImageView
18	android:id="@+id/imgComicCover"
19	android:layout_width="90dp"
20	android:layout_height="120dp"
21	android:scaleType="centerCrop"
22	app:shapeAppearanceOverlay="@style/RoundedImage"
23	android:contentDescription="@string/cd_comic_cover"/>
24	
25	<LinearLayout
26	android:layout_width="0dp"
27	android:layout_height="wrap_content"
28	android:layout_weight="1"
29	android:layout_marginStart="12dp"
30	android:orientation="vertical">
31	
32	<LinearLayout
33	android:layout_width="match_parent"
34	android:layout_height="wrap_content"
35	android:orientation="horizontal">
36	
37	<TextView
38	android:id="@+id/tvComicTitle"
39	android:layout_width="0dp"
40	android:layout_height="wrap_content"
41	android:layout_weight="1"
42	android:textSize="16sp"
43	android:textStyle="bold"
44	android:maxLines="1"
45	android:ellipsize="end"/>
46	
47	<TextView
48	android:id="@+id/tvComicYear"
49	android:layout_width="wrap_content"

```

50         android:layout_height="wrap_content"
51         android:textSize="12sp"/>
52
53     </LinearLayout>
54
55     <LinearLayout
56         android:layout_width="match_parent"
57         android:layout_height="wrap_content"
58         android:orientation="horizontal"
59         android:layout_marginTop="4dp">
60
61         <TextView
62             android:id="@+id/tvComicGenre"
63             android:layout_width="0dp"
64             android:layout_height="wrap_content"
65             android:layout_weight="1"
66             android:textSize="12sp"
67             android:textStyle="bold"/>
68
69         <TextView
70             android:id="@+id/tvComicAuthor"
71             android:layout_width="wrap_content"
72             android:layout_height="wrap_content"
73             android:textSize="12sp"/>
74
75     </LinearLayout>
76
77     <LinearLayout
78         android:layout_width="match_parent"
79         android:layout_height="wrap_content"
80         android:orientation="horizontal"
81         android:gravity="end"
82         android:layout_marginTop="8dp">
83
84         <Button
85             android:id="@+id/btnRead"
86             android:layout_width="wrap_content"
87             android:layout_height="wrap_content"
88             android:text="@string/btn_read"
89             android:layout_marginEnd="8dp"/>
90
91         <Button
92             android:id="@+id/btnDetail"
93             android:layout_width="wrap_content"
94             android:layout_height="wrap_content"
95             android:text="@string/btn_detail"/>
96
97     </LinearLayout>
98
99 </LinearLayout>
100 </LinearLayout>
101 </com.google.android.material.card.MaterialCardView>

```

item_comic_highlight.xml

Table 17. Source Code item_comic_highlight.xml

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <com.google.android.material.card.MaterialCardView
3     xmlns:android="http://schemas.android.com/apk/res/android"

```

```

4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      android:layout_width="160dp"
6      android:layout_height="240dp"
7      android:layout_marginEnd="16dp"
8      app:cardCornerRadius="16dp"
9      app:cardElevation="4dp">
10
11      <LinearLayout
12          android:layout_width="match_parent"
13          android:layout_height="match_parent"
14          android:orientation="vertical">
15
16          <ImageView
17              android:id="@+id/imgHighlightCover"
18              android:layout_width="match_parent"
19              android:layout_height="0dp"
20              android:layout_weight="1"
21              android:scaleType="centerCrop"
22              android:contentDescription="@string/cd_comic_cover"/>
23
24          <LinearLayout
25              android:layout_width="match_parent"
26              android:layout_height="wrap_content"
27              android:orientation="vertical"
28              android:padding="8dp">
29
30              <TextView
31                  android:id="@+id/tvHighlightTitle"
32                  android:layout_width="match_parent"
33                  android:layout_height="wrap_content"
34                  android:textStyle="bold"
35                  android:textSize="12sp"
36                  android:maxLines="1"
37                  android:ellipsize="end"/>
38
39              <TextView
40                  android:id="@+id/tvHighlightGenre"
41                  android:layout_width="match_parent"
42                  android:layout_height="wrap_content"
43                  android:textSize="11sp"
44                  android:maxLines="1"
45                  android:ellipsize="end"/>
46
47              <Button
48                  android:id="@+id/btnHighlightDetail"
49                  android:layout_width="match_parent"
50                  android:layout_height="wrap_content"
51                  android:layout_marginTop="4dp"
52                  android:text="@string/btn_detail"
53                  android:textSize="11sp"
54                  style="@style/Widget.Material3.Button"/>
55
56          </LinearLayout>
57      </LinearLayout>
58 </com.google.android.material.card.MaterialCardView>

```

COMPOSE

ComicDataSource.kt

Table 18. Source Code ComicDataSource.kt

1	package com.example.mycomicappcompose4.data
2	
3	import android.content.Context
4	import com.example.mycomicappcompose.R
5	
6	object ComicDataSource {
7	fun getComicList(context: Context) = listOf(
8	Comic(1, "Like Mother Like Daughter", "Thriller", "Carnby Kim", "2023",
9	context.getString(R.string.synopsis_like_mother),
10	R.drawable.lmld,
11	"https://www.webtoons.com/id/thriller/like-mother-like-
12	daughter/list?title_no=3622"),
13	Comic(2, "The Real Lesson", "Action", "Hyun", "2022",
14	context.getString(R.string.synopsis_real_lesson),
15	R.drawable.thereallesson,
16	"https://www.webtoons.com/id/action/the-real-
17	lesson/list?title_no=2423"),
18	Comic(3, "Wee!", "Slice of Life", "Soonkki", "2023",
19	context.getString(R.string.synopsis_wee),
20	R.drawable.wee,
21	"https://www.webtoons.com/id/slice-of-life/wee/list?title_no=3085"),
22	Comic(4, "The Villainess Reverses the Hourglass", "Romantic Fantasy",
23	"Sansobee", "2021",
24	context.getString(R.string.synopsis_villainess),
25	R.drawable.villainess,
26	"https://www.webtoons.com/id/romantic-fantasy/the-villainess-
27	reverses-hourglass/list?title_no=9517"),
28	Comic(5, "The Fantasie of a Stepmother", "Romantic Fantasy", "Rukyung",
29	"2020",
30	context.getString(R.string.synopsis_stepmother),
31	R.drawable.stepmother,
32	"https://www.webtoons.com/id/romantic-fantasy/the-fantasie-of-a-
33	stepmother/list?title_no=9773")
34	)
35	}

ComicDetailScreen.kt

Table 19. Source Code ComicDetailScreen.kt

1	package com.example.mycomicappcompose4
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import androidx.compose.foundation.verticalScroll
7	import androidx.compose.material3.Text
8	import androidx.compose.runtime.*
9	import androidx.compose.ui.Modifier
10	import androidx.compose.ui.layout.ContentScale
11	import androidx.compose.ui.platform.LocalContext
12	import androidx.compose.ui.res.painterResource
13	import androidx.compose.ui.res.stringResource

```

14 import androidx.compose.ui.text.font.FontWeight
15 import androidx.compose.ui.unit.dp
16 import androidx.compose.ui.unit.sp
17 import com.example.mycomicappcompose.R
18 @Composable
19 fun ComicDetailScreen(comicId: Int?, viewModel: ComicViewModel) {
20     val context = LocalContext.current
21     val comicList by viewModel.comicList.collectAsState()
22
23     LaunchedEffect(Unit) {
24         if (comicList.isEmpty()) {
25             viewModel.loadComics(context)
26         }
27     }
28
29     val comic = comicList.find { it.id == comicId }
30
31     if (comic != null) {
32         Column(
33             modifier = Modifier
34                 .fillMaxSize()
35                 .verticalScroll(rememberScrollState())
36         ) {
37             Image(
38                 painter = painterResource(id = comic.imageResId),
39                 contentDescription = comic.title,
40                 contentScale = ContentScale.Crop,
41                 modifier = Modifier
42                     .fillMaxWidth()
43                     .height(350.dp)
44             )
45             Column(modifier = Modifier.padding(16.dp)) {
46                 Row(
47                     modifier = Modifier.fillMaxWidth(),
48                     horizontalArrangement = Arrangement.SpaceBetween
49                 ) {
50                     Text(
51                         text = comic.title,
52                         fontWeight = FontWeight.Bold,
53                         fontSize = 24.sp,
54                         modifier = Modifier.weight(1f)
55                     )
56                     Text(
57                         text = comic.year,
58                         fontSize = 16.sp,
59                         fontWeight = FontWeight.Bold
60                     )
61                 }
62                 Spacer(modifier = Modifier.height(8.dp))
63                 Text(
64                     text = stringResource(R.string.label_genre) + comic.genre,
65                     fontSize = 14.sp
66                 )
67                 Text(
68                     text = stringResource(R.string.label_author) + comic.author,
69                     fontSize = 14.sp
70                 )
71                 Spacer(modifier = Modifier.height(12.dp))
72                 Text(
73                     text = stringResource(R.string.label_synopsis),
74                     fontWeight = FontWeight.Bold,
75                     fontSize = 16.sp
76                 )

```



```

77         Spacer(modifier = Modifier.height(4.dp))
78         Text(text = comic.synopsis, fontSize = 14.sp)
79     }
80 }
81 }
82 }

```

ComicListScreen.kt

Table 20. Source Code ComicListScreen.kt

```

1  package com.example.mycomicappcompose4
2
3  import android.content.Intent
4  import androidx.compose.foundation.Image
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.lazy.LazyColumn
8  import androidx.compose.foundation.lazy.LazyRow
9  import androidx.compose.foundation.lazy.items
10 import androidx.compose.foundation.shape.RoundedCornerShape
11 import androidx.compose.material.icons.Icons
12 import androidx.compose.material.icons.filled.Language
13 import androidx.compose.material3.*
14 import androidx.compose.runtime.*
15 import androidx.compose.ui.Alignment
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.draw.clip
18 import androidx.compose.ui.layout.ContentScale
19 import androidx.compose.ui.platform.LocalContext
20 import androidx.compose.ui.res.painterResource
21 import androidx.compose.ui.res.stringResource
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.text.style.TextOverflow
24 import androidx.compose.ui.unit.dp
25 import androidx.compose.ui.unit.sp
26 import androidx.core.net.toUri
27 import androidx.navigation.NavController
28 import com.example.mycomicappcompose.R
29 import com.example.mycomicappcompose4.data.Comic
30
31 @Composable
32 fun ComicListScreen(navController: NavController, viewModel: ComicViewModel) {
33     val context = LocalContext.current
34     val comicList by viewModel.comicList.collectAsState()
35     val selectedComicId by viewModel.selectedComicId.collectAsState()
36     val openUrlEvent by viewModel.openUrlEvent.collectAsState()
37
38     LaunchedEffect(Unit) {
39         viewModel.loadComics(context)
40     }
41
42     LaunchedEffect(selectedComicId) {
43         selectedComicId?.let { id ->
44             viewModel.onNavigatedToDetail(id)
45             navController.navigate("detail/$id")
46         }
47     }
48
49     LaunchedEffect(openUrlEvent) {

```

```

50         openUrlEvent?.let { url ->
51             val intent = Intent(Intent.ACTION_VIEW, url.toUri())
52             context.startActivity(intent)
53             viewModel.onUrlOpened()
54         }
55     }
56
57     LazyColumn(
58         modifier = Modifier.fillMaxSize(),
59         contentPadding = PaddingValues(16.dp)
60     ) {
61         item {
62             Row(
63                 modifier = Modifier.fillMaxWidth(),
64                 horizontalArrangement = Arrangement.SpaceBetween,
65                 verticalAlignment = Alignment.CenterVertically
66             ) {
67                 Text(
68                     text = stringResource(R.string.title_highlight),
69                     fontWeight = FontWeight.Bold,
70                     fontSize = 20.sp
71                 )
72                 IconButton(onClick = { navController.navigate("language") }) {
73                     Icon(
74                         imageVector = Icons.Filled.Language,
75                         contentDescription = "Language"
76                     )
77                 }
78             }
79         }
80
81         item {
82             LazyRow(
83                 horizontalArrangement = Arrangement.spacedBy(16.dp),
84                 modifier = Modifier.padding(bottom = 24.dp)
85             ) {
86                 items(comicList) { comic ->
87                     ComicHighlightCard(
88                         comic = comic,
89                         onDetailClick = { viewModel.onDetailClicked(comic.id) }
90                     )
91                 }
92             }
93         }
94
95         item {
96             Text(
97                 text = stringResource(R.string.title_all_comics),
98                 fontWeight = FontWeight.Bold,
99                 fontSize = 20.sp,
100                 modifier = Modifier.padding(bottom = 12.dp)
101             )
102         }
103
104         items(comicList) { comic ->
105             ComicItem(
106                 comic = comic,
107                 onDetailClick = { viewModel.onDetailClicked(comic.id) },
108                 comic.title) {
109                 viewModel.onReadClicked(comic.webtoonUrl,
110                 Spacer(modifier = Modifier.height(16.dp))
111             }

```

```

112     }
113 }
114
115 @Composable
116 fun ComicHighlightCard(comic: Comic, onDetailClick: () -> Unit) {
117     Card(
118         shape = RoundedCornerShape(16.dp),
119         elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
120         modifier = Modifier
121             .width(160.dp)
122             .height(240.dp)
123             .clickable { onDetailClick() }
124     ) {
125         Column(modifier = Modifier.fillMaxSize()) {
126             Image(
127                 painter = painterResource(id = comic.imageResId),
128                 contentDescription = comic.title,
129                 contentScale = ContentScale.Crop,
130                 modifier = Modifier
131                     .fillMaxWidth()
132                     .weight(1f)
133             )
134             Column(modifier = Modifier.padding(8.dp)) {
135                 Text(
136                     text = comic.title,
137                     fontWeight = FontWeight.Bold,
138                     fontSize = 12.sp,
139                     maxLines = 1,
140                     overflow = TextOverflow.Ellipsis
141                 )
142                 Text(
143                     text = comic.genre,
144                     fontSize = 11.sp,
145                     maxLines = 1,
146                     overflow = TextOverflow.Ellipsis
147                 )
148                 Spacer(modifier = Modifier.height(4.dp))
149                 Button(
150                     onClick = onDetailClick,
151                     modifier = Modifier.fillMaxWidth(),
152                     contentPadding = PaddingValues(0.dp)
153                 ) {
154                     Text(stringResource(R.string.btn_detail), fontSize = 11.sp)
155                 }
156             }
157         }
158     }
159 }
160
161 @Composable
162 fun ComicItem(comic: Comic, onDetailClick: () -> Unit, onReadClick: () -> Unit) {
163     Card(
164         shape = RoundedCornerShape(16.dp),
165         elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
166         modifier = Modifier
167             .fillMaxWidth()
168             .clickable { onDetailClick() }
169     ) {
170         Row(modifier = Modifier.padding(12.dp)) {
171             Image(
172                 painter = painterResource(id = comic.imageResId),
173                 contentDescription = comic.title,
174                 contentScale = ContentScale.Crop,

```

```

175         modifier = Modifier
176             .width(90.dp)
177             .height(120.dp)
178             .clip(RoundedCornerShape(12.dp))
179     )
180     Spacer(modifier = Modifier.width(12.dp))
181     Column(modifier = Modifier.weight(1f)) {
182         Row(
183             modifier = Modifier.fillMaxWidth(),
184             horizontalArrangement = Arrangement.SpaceBetween
185         ) {
186             Text(
187                 text = comic.title,
188                 fontWeight = FontWeight.Bold,
189                 fontSize = 16.sp,
190                 maxLines = 1,
191                 overflow = TextOverflow.Ellipsis,
192                 modifier = Modifier.weight(1f)
193             )
194             Text(text = comic.year, fontSize = 12.sp)
195         }
196         Row(
197             modifier = Modifier.fillMaxWidth(),
198             horizontalArrangement = Arrangement.SpaceBetween
199         ) {
200             Text(
201                 text = stringResource(R.string.label_genre) +
comic.genre,
202                 fontWeight = FontWeight.Bold,
203                 fontSize = 12.sp,
204                 modifier = Modifier.weight(1f)
205             )
206             Text(text = comic.author, fontSize = 12.sp)
207         }
208         Spacer(modifier = Modifier.height(8.dp))
209         Row(
210             horizontalArrangement = Arrangement.End,
211             modifier = Modifier.fillMaxWidth()
212         ) {
213             Button(
214                 onClick = onReadClick,
215                 modifier = Modifier.padding(end = 8.dp)
216             ) {
217                 Text(stringResource(R.string.btn_read))
218             }
219             Button(onClick = onDetailClick) {
220                 Text(stringResource(R.string.btn_detail))
221             }
222         }
223     }
224 }
225 }
226 }

```

ComicViewModel.kt

Table 21. Source Code ComicViewModel.kt

```

1 package com.example.mycomicappcompose4
2

```

```

3 import android.content.Context
4 import androidx.lifecycle.ViewModel
5 import com.example.mycomicappcompose4.data.Comic
6 import com.example.mycomicappcompose4.data.ComicDataSource
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import timber.log.Timber
10
11 class ComicViewModel(private val appName: String) : ViewModel() {
12
13     private val _comicList = MutableStateFlow<List<Comic>>(emptyList())
14     val comicList: StateFlow<List<Comic>> = _comicList
15
16     private val _selectedComicId = MutableStateFlow<Int?>(null)
17     val selectedComicId: StateFlow<Int?> = _selectedComicId
18
19     private val _openUrlEvent = MutableStateFlow<String?>(null)
20     val openUrlEvent: StateFlow<String?> = _openUrlEvent
21
22     fun loadComics(context: Context) {
23         val list = ComicDataSource.getComicList(context)
24         _comicList.value = list
25         Timber.d("[${appName}] Data loaded: ${list.size} comics masuk ke dalam
26 list")
27         list.forEach { comic ->
28             Timber.d("[${appName}] Item:   ${comic.title}   |   ${comic.genre}   |
29 ${comic.year}")
30         }
31
32         fun onDetailClicked(comicId: Int) {
33             val comic = _comicList.value.find { it.id == comicId }
34             Timber.d("[${appName}] Tombol Detail ditekan → ${comic?.title} | Genre:
35 ${comic?.genre}")
36             _selectedComicId.value = comicId
37         }
38
39         fun onReadClicked(url: String, title: String) {
40             Timber.d("[${appName}] Tombol Read ditekan → URL: $url | Judul: $title")
41             _openUrlEvent.value = url
42         }
43
44         fun onUrlOpened() {
45             _openUrlEvent.value = null
46         }
47
48         fun onNavigatedToDetail(comicId: Int) {
49             val comic = _comicList.value.find { it.id == comicId }
50             Timber.d("[${appName}] Berpindah ke Detail → ${comic?.title} | Author:
51 ${comic?.author} | Year: ${comic?.year}")
52             _selectedComicId.value = null
53         }
54     }
55 }

```

ComicViewModelFactory.kt

Table 22. Source Code ComicViewModelFactory.kt

```

1 package com.example.mycomicappcompose4
2

```

```

3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.ViewModelProvider
5
6  class ComicViewModelFactory(private val appName: String) :
  ViewModelProvider.Factory {
7      override fun <T : ViewModel> create(modelClass: Class<T>): T {
8          if (modelClass.isAssignableFrom(ComicViewModel::class.java)) {
9              @Suppress("UNCHECKED_CAST")
10             return ComicViewModel(appName) as T
11         }
12         throw IllegalArgumentException("Unknown ViewModel class")
13     }
14 }

```

LanguageScreen.kt

Table 23. Source Code LanguageScreen.kt

```

1  package com.example.mycomicappcompose4
2
3  import android.content.res.Configuration
4  import androidx.compose.foundation.layout.*
5  import androidx.compose.material3.Button
6  import androidx.compose.material3.Text
7  import androidx.compose.runtime.Composable
8  import androidx.compose.ui.Alignment
9  import androidx.compose.ui.Modifier
10 import androidx.compose.ui.platform.LocalContext
11 import androidx.compose.ui.res.stringResource
12 import androidx.compose.ui.text.font.FontWeight
13 import androidx.compose.ui.unit.dp
14 import androidx.compose.ui.unit.sp
15 import androidx.navigation.NavController
16 import com.example.mycomicappcompose.R
17 import java.util.Locale
18
19 @Composable
20 fun LanguageScreen(navController: NavController) {
21     val context = LocalContext.current
22
23     Column(
24         modifier = Modifier
25             .fillMaxSize()
26             .padding(32.dp),
27         verticalArrangement = Arrangement.Center,
28         horizontalAlignment = Alignment.CenterHorizontally
29     ) {
30         Text(
31             text = stringResource(R.string.title_language),
32             fontWeight = FontWeight.Bold,
33             fontSize = 22.sp,
34             modifier = Modifier.padding(bottom = 32.dp)
35         )
36         Button(
37             onClick = {
38                 val locale = Locale("en")
39                 Locale.setDefault(locale)
40                 val config = Configuration(context.resources.configuration)
41                 config.setLocale(locale)

```

```

42         context.resources.updateConfiguration(config,
context.resources.displayMetrics)
43         navController.popBackStack()
44     },
45     modifier = Modifier
46         .fillMaxWidth()
47         .padding(bottom = 16.dp)
48     ) {
49         Text(stringResource(R.string.btn_english))
50     }
51     Button(
52         onClick = {
53             val locale = Locale("in")
54             Locale.setDefault(locale)
55             val config = Configuration(context.resources.configuration)
56             config.setLocale(locale)
57             context.resources.updateConfiguration(config,
context.resources.displayMetrics)
58             navController.popBackStack()
59         },
60         modifier = Modifier.fillMaxWidth()
61     ) {
62         Text(stringResource(R.string.btn_indonesian))
63     }
64 }
65 }

```

MainActivity.kt

Table 24. Source Code MainActivity.kt

```

1 package com.example.mycomicappcompose4
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.runtime.Composable
7 import androidx.lifecycle.viewmodel.compose.viewModel
8 import androidx.navigation.NavType
9 import androidx.navigation.compose.NavHost
10 import androidx.navigation.compose.composable
11 import androidx.navigation.compose.rememberNavController
12 import androidx.navigation.navArgument
13
14 class MainActivity : ComponentActivity() {
15     override fun onCreate(savedInstanceState: Bundle?) {
16         super.onCreate(savedInstanceState)
17         setContent {
18             ComicApp()
19         }
20     }
21 }
22
23 @Composable
24 fun ComicApp() {
25     val navController = rememberNavController()
26     val factory = ComicViewModelFactory(appName = "MyComicAppCompose4")
27     val viewModel: ComicViewModel = viewModel(factory = factory)
28
29     NavHost(navController = navController, startDestination = "home") {

```

```

30         composable("home") {
31             ComicListScreen(
32                 navController = navController,
33                 viewModel = viewModel
34             )
35         }
36         composable(
37             route = "detail/{comicId}",
38             arguments = listOf(navArgument("comicId") { type = NavType.IntType })
39         ) { backStackEntry ->
40             val comicId = backStackEntry.arguments?.getInt("comicId")
41             ComicDetailScreen(
42                 comicId = comicId,
43                 viewModel = viewModel
44             )
45         }
46         composable("language") {
47             LanguageScreen(navController = navController)
48         }
49     }
50 }

```

MyApp.kt

Table 25. Source Code MyApp.kt

```

1 package com.example.mycomicappcompose4
2
3 import android.app.Application
4 import timber.log.Timber
5
6 class MyApp : Application() {
7     override fun onCreate() {
8         super.onCreate()
9         Timber.plant(Timber.DebugTree())
10    }
11 }

```

Comic.kt

Table 26. Source Code Comic.kt

```

1 package com.example.mycomicappcompose4.data
2
3 data class Comic(
4     val id: Int,
5     val title: String,
6     val genre: String,
7     val author: String,
8     val year: String,
9     val synopsis: String,
10    val imageResId: Int,
11    val webtoonUrl: String
12 )

```

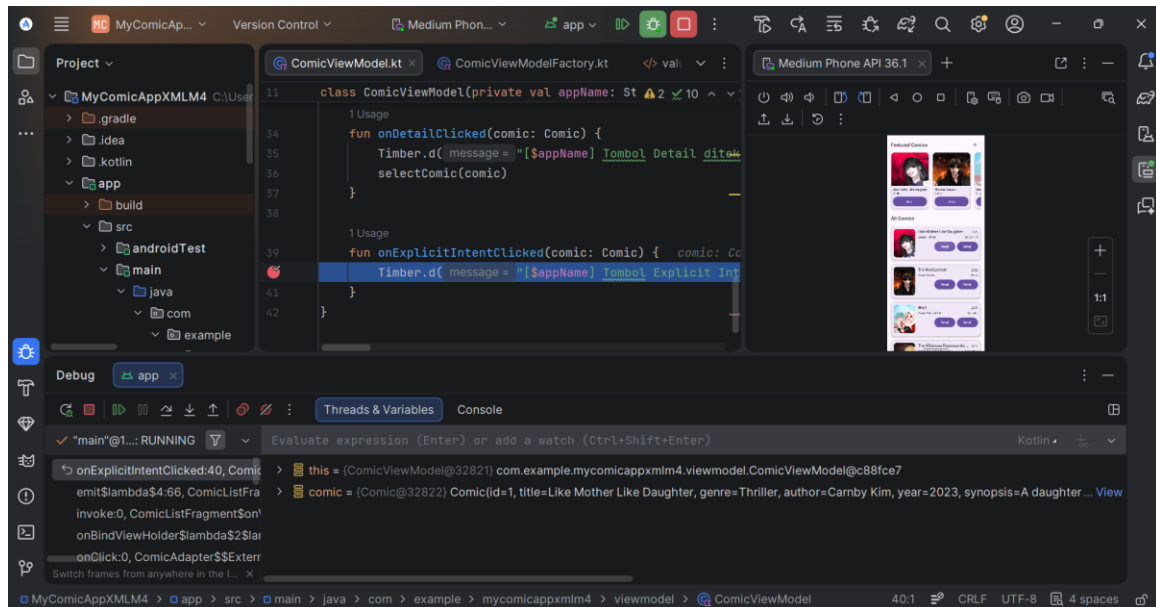

ComicDataSource.kt

Table 27. Source Code ComicDataSource.kt

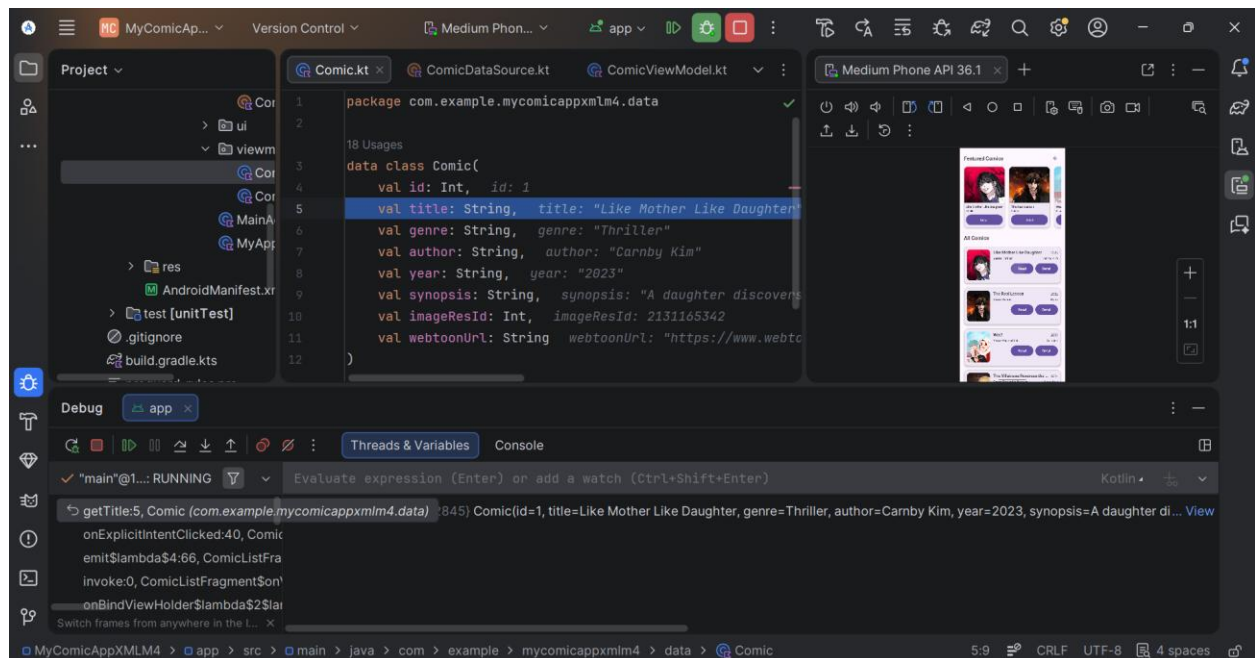
```
1 package com.example.mycomicappcompose4.data
2
3 import android.content.Context
4 import com.example.mycomicappcompose.R
5
6 object ComicDataSource {
7     fun getComicList(context: Context) = listOf(
8         Comic(1, "Like Mother Like Daughter", "Thriller", "Carnby Kim", "2023",
9             context.getString(R.string.synopsis_like_mother),
10             R.drawable.lmld,
11             "https://www.webtoons.com/id/thriller/like-mother-like-
12 daughter/list?title_no=3622"),
13         Comic(2, "The Real Lesson", "Action", "Hyun", "2022",
14             context.getString(R.string.synopsis_real_lesson),
15             R.drawable.thereallesson,
16             "https://www.webtoons.com/id/action/the-real-
17 lesson/list?title_no=2423"),
18         Comic(3, "Wee!", "Slice of Life", "Soonkki", "2023",
19             context.getString(R.string.synopsis_wee),
20             R.drawable.wee,
21             "https://www.webtoons.com/id/slice-of-life/wee/list?title_no=3085"),
22         Comic(4, "The Villainess Reverses the Hourglass", "Romantic Fantasy",
23             "Sansobee", "2021",
24             context.getString(R.string.synopsis_villainess),
25             R.drawable.villainess,
26             "https://www.webtoons.com/id/romantic-fantasy/the-villainess-
27 reverses-hourglass/list?title_no=9517"),
28         Comic(5, "The Fantasie of a Stepmother", "Romantic Fantasy", "Rukyung",
29             "2020",
30             context.getString(R.string.synopsis_stepmother),
31             R.drawable.stepmother,
32             "https://www.webtoons.com/id/romantic-fantasy/the-fantasie-of-a-
33 stepmother/list?title_no=9773")
34     )
35 }
```

B. Output Program

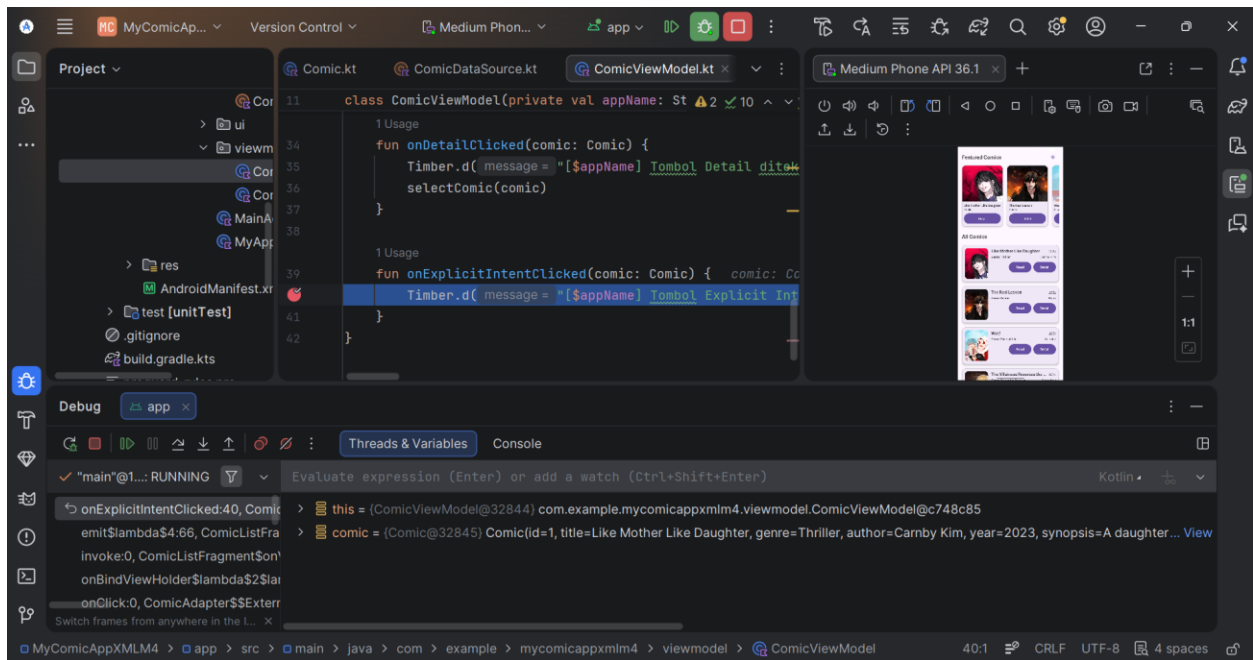
ComicAppXML



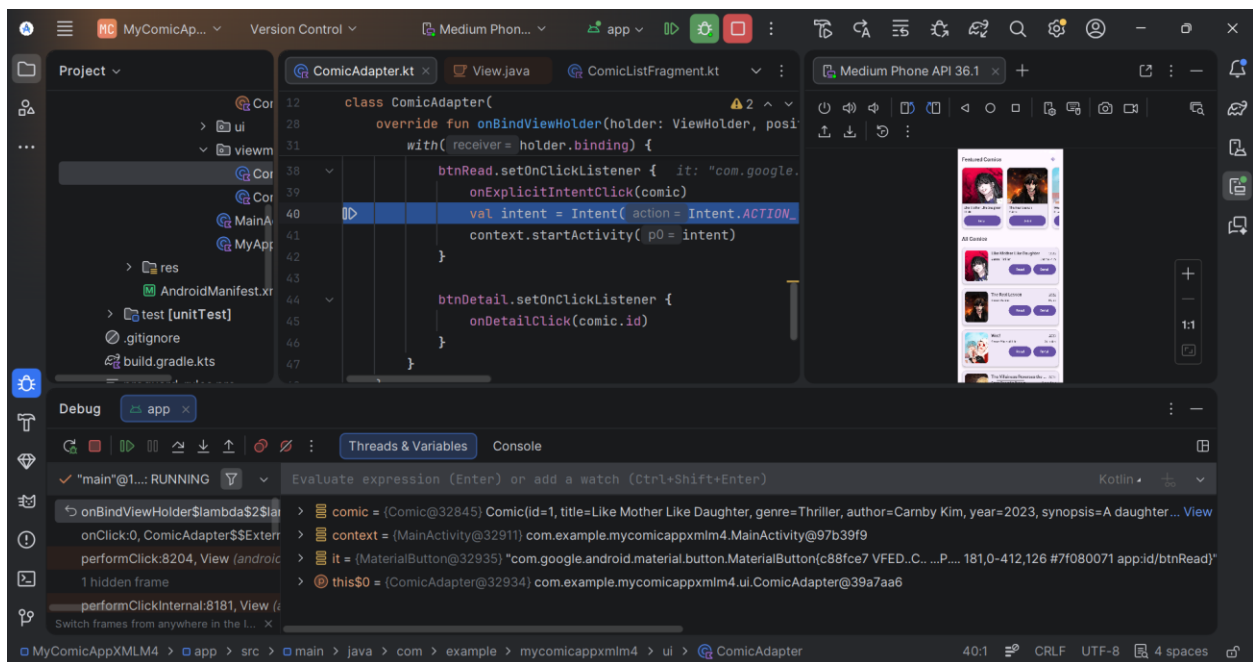
Gambar 1. Screenshot Hasil Test Debugging Soal 1 XML



Gambar 2. Screenshot Hasil Test Debugging Soal 1 XML

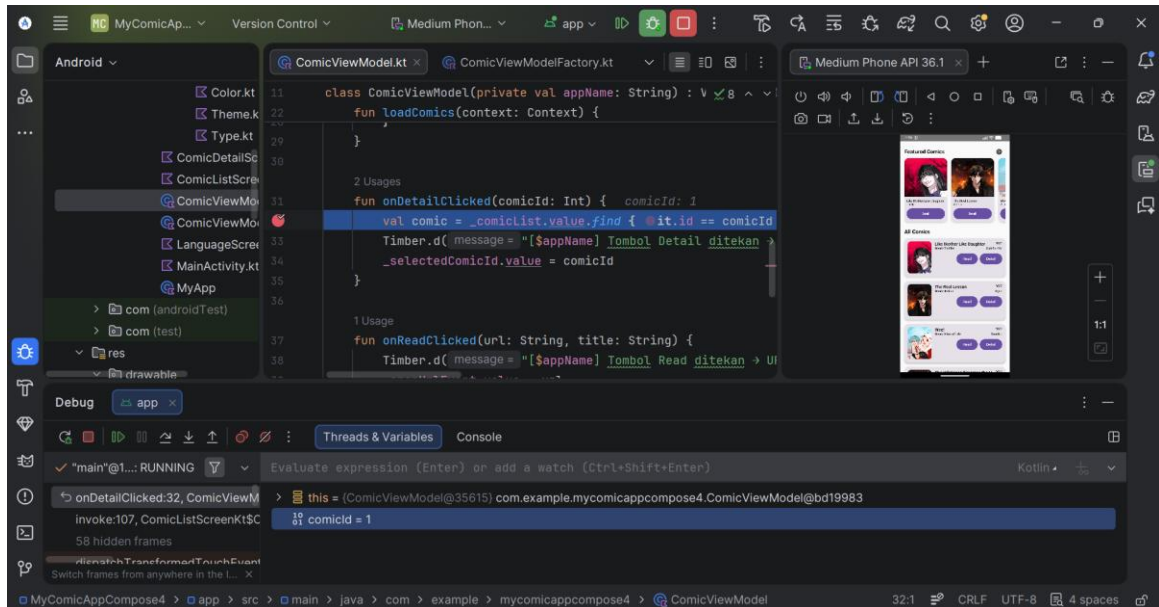


Gambar 3. Screenshot Hasil Test Debugging Soal 1 XML

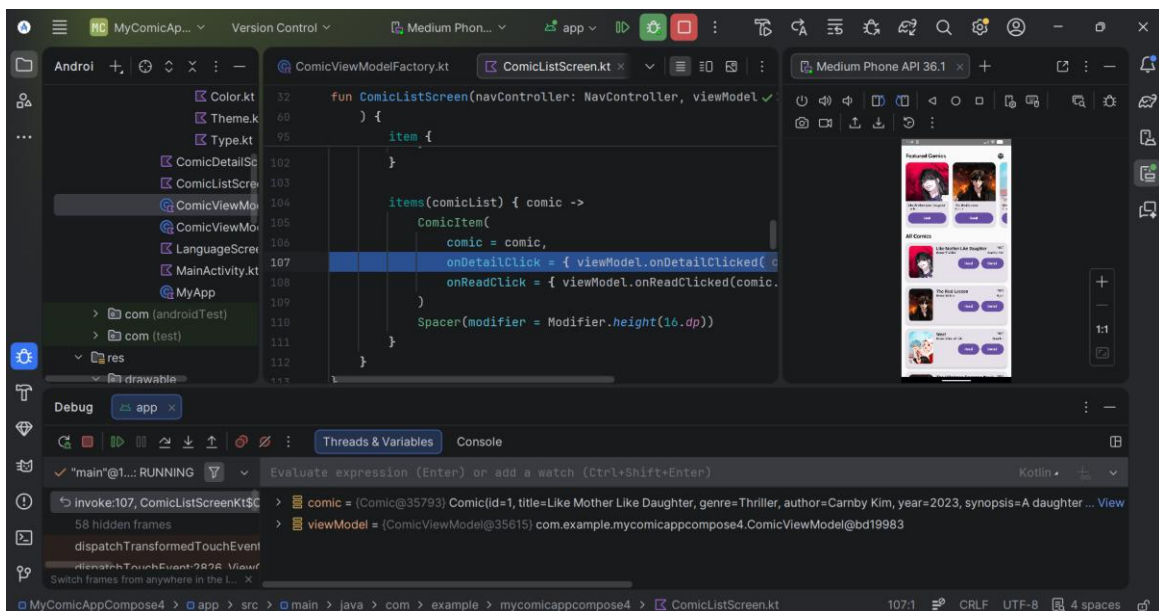


Gambar 4. Screenshot Hasil Test Debugging Soal 1 XML

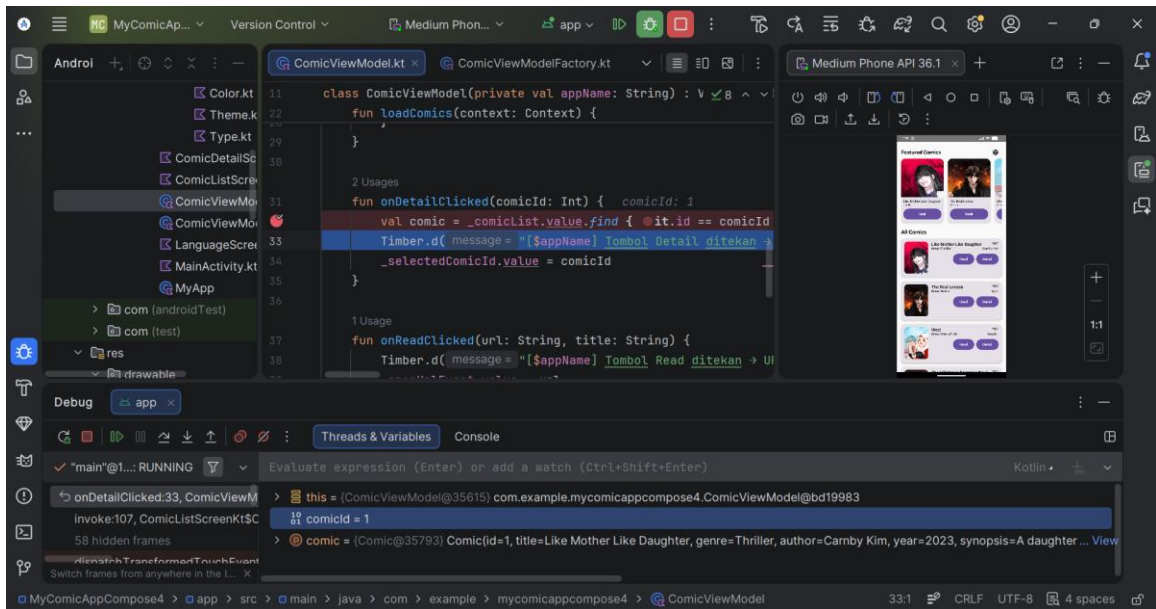
ComicAppCompose



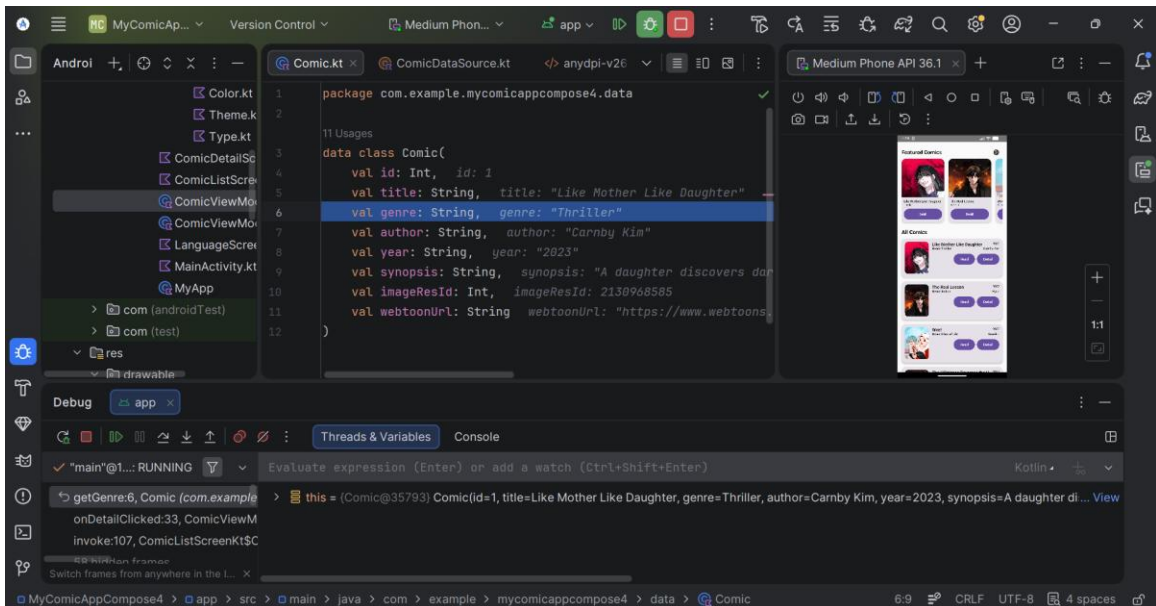
Gambar 5. Screenshot Hasil Test Debugging Soal 1 Compose



Gambar 6. Screenshot Hasil Test Debugging Soal 1 Compose



Gambar 7. Screenshot Hasil Test Debugging Soal 1 Compose



Gambar 8. Screenshot Hasil Test Debugging Soal 1 Compose

C. Pembahasan

XML

MainActivity.kt (XML)

Class 'MainActivity' mewarisi 'AppCompatActivity' dan menggunakan ViewBinding melalui properti 'binding' bertipe 'ActivityMainBinding'. Di dalam 'onCreate()', layout di-inflate menggunakan 'ActivityMainBinding.inflate(layoutInflater)' dan di-set sebagai content view melalui 'setContentView(binding.root)'.

Comic.kt

File 'Comic.kt' mendefinisikan data class 'Comic' yang berada di dalam package 'com.example.mycomicappxmlm4.data'. Data class ini memiliki delapan properti yaitu 'id' bertipe 'Int', 'title', 'genre', 'author', 'year', 'synopsis' bertipe 'String', 'imageResId' bertipe 'Int', dan 'webtoonUrl' bertipe 'String'. Data class ini berfungsi sebagai model data yang merepresentasikan satu item komik dalam aplikasi.

ComicDataSource.kt

File 'ComicDataSource.kt' berfungsi sebagai sumber data statis untuk daftar komik. Object 'ComicDataSource' memiliki fungsi 'getComicList()' yang menerima parameter 'context' bertipe 'Context' dan mengembalikan list berisi lima objek 'Comic'. Setiap objek 'Comic' diinisialisasi dengan data seperti judul, genre, author, tahun, sinopsis yang diambil menggunakan 'context.getString()', resource gambar menggunakan 'R.drawable', dan URL webtoon masing-masing komik.

ComicViewModel.kt

Class 'ComicViewModel' mewarisi 'ViewModel' dan menerima dua parameter yaitu 'appName' bertipe 'String' dan 'context' bertipe 'Context'. Data list komik dikelola menggunakan 'MutableStateFlow' yang di-expose sebagai 'StateFlow' melalui properti 'comicList', dan comic yang dipilih dikelola melalui properti 'selectedComic'. Di dalam blok 'init', fungsi 'loadComics()' dipanggil secara otomatis untuk memuat data dari 'ComicDataSource.getComicList()' dan menyimpannya ke '_comicList.value'. Timber digunakan untuk mencatat log saat data berhasil dimuat ke dalam list, saat tombol Detail ditekan melalui fungsi 'onDetailClicked()', dan saat tombol Explicit Intent ditekan melalui fungsi 'onExplicitIntentClicked()'.

ComicViewModelFactory.kt

Class 'ComicViewModelFactory' mengimplementasikan 'ViewModelProvider.Factory' dan menerima dua parameter yaitu 'appName' bertipe 'String' dan 'context' bertipe 'Context'. Di dalam fungsi 'create()', dilakukan pengecekan apakah 'modelClass' sesuai dengan 'ComicViewModel::class.java', dan jika sesuai maka instance 'ComicViewModel' dibuat dengan meneruskan 'appName' dan 'context' sebagai argumen menggunakan '@SuppressWarnings("UNCHECKED_CAST")' untuk menghindari peringatan cast yang tidak bisa diverifikasi.

ComicAdapter.kt

Class 'ComicAdapter' mewarisi 'RecyclerView.Adapter' dan menerima tiga parameter yaitu 'comics' bertipe 'List<Comic>', 'onDetailClick' sebagai lambda dengan parameter 'Int', dan 'onExplicitIntentClick' sebagai lambda dengan parameter 'Comic'. Inner class 'ViewHolder' menyimpan referensi ke 'ItemComicBinding'. Di dalam 'onBindViewHolder()', data komik di-bind ke view meliputi 'imgComicCover' menggunakan 'setImageResource()', 'tvComicTitle', 'tvComicYear', 'tvComicGenre', dan 'tvComicAuthor'. Tombol 'btnRead' memanggil 'onExplicitIntentClick' dan membuka URL webtoon menggunakan 'Intent.ACTION_VIEW', sedangkan tombol 'btnDetail' memanggil 'onDetailClick' dengan 'comic.id' sebagai argumen.

ComicHighlightAdapter.kt

Class 'ComicHighlightAdapter' mewarisi 'RecyclerView.Adapter' dan menerima dua parameter yaitu 'comics' bertipe 'List<Comic>' dan 'onDetailClick' sebagai lambda dengan parameter 'Int'. Inner class 'ViewHolder' menyimpan referensi ke 'ItemComicHighlightBinding'. Di dalam 'onBindViewHolder()', data komik di-bind ke view meliputi 'imgHighlightCover' menggunakan 'setImageResource()', 'tvHighlightTitle', dan 'tvHighlightGenre'. Tombol 'btnHighlightDetail' memanggil 'onDetailClick' dengan 'comic.id' sebagai argumen.

ComicListFragment.kt

Class 'ComicListFragment' mewarisi 'Fragment' dan menggunakan ViewBinding melalui 'FragmentComicListBinding' dengan pola '_binding' nullable untuk menghindari memory leak di 'onDestroyView()'. ViewModel dibuat menggunakan 'viewModels' dengan 'ComicViewModelFactory' yang menerima parameter 'appName' bertipe 'String' dan 'context'. RecyclerView 'rvHighlight' dikonfigurasi dengan 'LinearLayoutManager' horizontal, sedangkan

'rvComicList' menggunakan 'LinearLayoutManager' vertikal. Data dikumpulkan dari 'viewModel.comicList' menggunakan 'lifecycleScope.launch' dan 'collect' untuk memperbarui 'ComicHighlightAdapter' dan 'ComicAdapter'. Event onClick tombol Detail dikelola melalui lambda 'onDetailClick' yang memanggil 'viewModel.onDetailClicked()' dan memicu navigasi ke halaman detail menggunakan 'findNavController().navigate()', sementara event onClick tombol Read dikelola melalui lambda 'onExplicitIntentClick' yang memanggil 'viewModel.onExplicitIntentClicked()'. Tombol 'btnLanguage' memicu navigasi ke 'LanguageFragment' menggunakan 'findNavController().navigate()'.

ComicDetailFragment.kt

Class 'ComicDetailFragment' mewarisi 'Fragment' dan menggunakan ViewBinding melalui 'FragmentComicDetailBinding' dengan pola '_binding' nullable untuk menghindari memory leak di 'onDestroyView()'. ViewModel dibuat menggunakan 'viewModels' dengan 'ComicViewModelFactory' yang menerima parameter 'appName' bertipe 'String' dan 'context'. 'comicId' diterima melalui 'arguments?.getInt("comicId")' yang dikirim dari fragment sebelumnya menggunakan Bundle. Data komik di-collect dari 'viewModel.comicList' menggunakan 'lifecycleScope.launch' dan 'collect', kemudian difilter menggunakan 'find' berdasarkan 'comicId' untuk menemukan komik yang sesuai. Saat data diterima, Timber mencatat log dengan pesan yang menyertakan judul komik melalui 'Timber.d()', lalu 'viewModel.selectComic()' dipanggil dan data ditampilkan ke view binding meliputi 'imgDetailCover', 'tvDetailTitle', 'tvDetailGenre', 'tvDetailAuthor', 'tvDetailYear', dan 'tvDetailSynopsis'.

LanguageFragment.kt

Class 'LanguageFragment' mewarisi 'Fragment' dan menggunakan ViewBinding melalui 'FragmentLanguageBinding' dengan pola '_binding' nullable untuk menghindari memory leak di 'onDestroyView()'. Tombol 'btnEnglish' memanggil fungsi 'setLocale()' dengan parameter "'en'", sedangkan tombol 'btnIndonesian' memanggil 'setLocale()' dengan parameter "'in'". Di dalam fungsi 'setLocale()', 'Locale' baru dibuat dan diterapkan menggunakan 'Locale.setDefault()', kemudian 'Configuration' diperbarui menggunakan 'resources.updateConfiguration()', dan activity di-restart menggunakan 'requireActivity().recreate()' agar perubahan bahasa diterapkan ke seluruh tampilan.

MyApplication.kt

Class 'MyApplication' mewarisi 'Application' dan di dalam 'onCreate()' memanggil 'Timber.plant(Timber.DebugTree())' untuk menginisialisasi library Timber agar sistem logging aktif sejak awal aplikasi berjalan.

activity_main.xml

File 'activity_main.xml' mendefinisikan layout utama aplikasi menggunakan 'ConstraintLayout' sebagai root layout dengan lebar dan tinggi 'match_parent'. Di dalamnya terdapat 'FragmentManager' dengan id 'nav_host_fragment' yang berfungsi sebagai wadah navigasi antar fragment. Komponen ini menggunakan 'androidx.navigation.fragment.NavHostFragment' sebagai 'android:name', dengan atribut 'app:defaultNavHost="true"' agar dapat menangani tombol back, dan 'app:navGraph="@navigation/nav_graph"' untuk menghubungkan ke grafik navigasi yang telah didefinisikan.

fragment_comic_detail.xml

File 'fragment_comic_detail.xml' mendefinisikan layout halaman detail komik menggunakan 'ScrollView' sebagai root layout agar konten dapat di-scroll. Di dalamnya terdapat 'LinearLayout' vertikal yang berisi 'ImageView' dengan id 'imgDetailCover' berukuran tinggi '350dp' dan 'scaleType' 'centerCrop' untuk menampilkan cover komik. Di bawahnya terdapat 'LinearLayout' vertikal dengan padding '16dp' yang berisi beberapa 'TextView' yaitu 'tvDetailTitle' dengan ukuran teks '24sp' dan style bold, 'tvDetailGenre', 'tvDetailAuthor', dan 'tvDetailYear' dengan ukuran teks '14sp', serta 'tvDetailSynopsisLabel' dengan ukuran teks '16sp' dan style bold, dan 'tvDetailSynopsis' untuk menampilkan isi sinopsis komik.

fragment_comic_list.xml

File 'fragment_comic_list.xml' mendefinisikan layout halaman daftar komik menggunakan 'NestedScrollView' sebagai root layout. Di dalamnya terdapat 'LinearLayout' vertikal dengan padding '16dp'. Bagian atas terdapat 'LinearLayout' horizontal yang berisi 'TextView' untuk judul seksi highlight dengan teks dari string resource 'title_highlight' dan 'MaterialButton' dengan id 'btnLanguage' yang menggunakan style icon button untuk navigasi ke halaman bahasa. Di bawahnya terdapat 'RecyclerView' dengan id 'rvHighlight' yang dikonfigurasi horizontal untuk menampilkan highlight komik, diikuti 'TextView' untuk judul seksi semua komik dengan teks dari 'title_all_comics', dan 'RecyclerView' dengan id 'rvComicList' dengan atribut 'nestedScrollingEnabled="false"' untuk menampilkan seluruh daftar komik secara vertikal.

fragment_language.xml

File 'fragment_language.xml' mendefinisikan layout halaman pemilihan bahasa menggunakan 'LinearLayout' vertikal sebagai root layout dengan atribut 'gravity="center"' dan padding '32dp' agar konten berada di tengah layar. Di dalamnya terdapat 'TextView' untuk judul halaman dengan ukuran teks '22sp' dan style bold, diikuti dua buah 'Button' yaitu 'btnEnglish' dengan teks dari string resource 'btn_english' untuk memilih bahasa Inggris, dan 'btnIndonesian' dengan teks dari string resource 'btn_indonesian' untuk memilih bahasa Indonesia.

item_comic.xml

File 'item_comic.xml' mendefinisikan layout untuk setiap item pada RecyclerView daftar komik menggunakan 'MaterialCardView' sebagai root dengan sudut melengkung 'cardCornerRadius' '16dp' dan elevasi '4dp'. Di dalamnya terdapat 'LinearLayout' horizontal dengan padding '12dp' yang berisi 'ShapeableImageView' dengan id 'imgComicCover' berukuran '90dp' x '120dp' dengan style sudut melengkung dari 'RoundedImage', dan 'LinearLayout' vertikal dengan 'layout_weight="1"' yang berisi informasi komik. Informasi komik mencakup 'LinearLayout' horizontal pertama yang berisi 'tvComicTitle' dengan style bold dan 'tvComicYear', 'LinearLayout' horizontal kedua yang berisi 'tvComicGenre' dengan style bold dan 'tvComicAuthor', serta 'LinearLayout' horizontal ketiga dengan 'gravity="end"' yang berisi tombol 'btnRead' dan 'btnDetail'.

item_comic_highlight.xml

File 'item_comic_highlight.xml' mendefinisikan layout untuk setiap item pada RecyclerView highlight komik menggunakan 'MaterialCardView' sebagai root dengan lebar '160dp', tinggi '240dp', sudut melengkung 'cardCornerRadius' '16dp', dan elevasi '4dp'. Di dalamnya terdapat 'LinearLayout' vertikal yang berisi 'ImageView' dengan id 'imgHighlightCover' menggunakan 'layout_weight="1"' agar gambar mengisi sisa ruang yang tersedia dengan 'scaleType' 'centerCrop', dan 'LinearLayout' vertikal dengan padding '8dp' yang berisi 'tvHighlightTitle' dengan style bold dan ukuran teks '12sp', 'tvHighlightGenre' dengan ukuran teks '11sp', serta tombol 'btnHighlightDetail' dengan style 'Widget.Material3.Button' dan ukuran teks '11sp'.

COMPOSE

MainActivity.kt

Class 'MainActivity' mewarisi 'ComponentActivity' dan di dalam 'onCreate()' memanggil 'setContent' untuk menampilkan fungsi composable 'ComicApp()'. Fungsi composable 'ComicApp()' membuat 'NavController' menggunakan 'rememberNavController()', membuat instance 'ComicViewModelFactory' dengan parameter 'appName' bernilai "'MyComicAppCompose4'", dan membuat 'viewModel' menggunakan fungsi 'viewModel()' dengan factory tersebut. Navigasi dikelola menggunakan 'NavHost' dengan tiga route yaitu "'home'" yang menampilkan 'ComicListScreen', "'detail/{comicId}'" dengan argumen 'comicId' bertipe 'NavType.IntType' yang menampilkan 'ComicDetailScreen', dan "'language'" yang menampilkan 'LanguageScreen'.

MyApp.kt

Class 'MyApp' mewarisi 'Application' dan di dalam 'onCreate()' memanggil 'Timber.plant(Timber.DebugTree())' untuk menginisialisasi library Timber agar sistem logging aktif sejak awal aplikasi berjalan.

Comic.kt

File 'Comic.kt' mendefinisikan data class 'Comic' yang berada di dalam package 'com.example.mycomicappcompose4.data'. Data class ini memiliki delapan properti yaitu 'id' bertipe 'Int', 'title', 'genre', 'author', 'year', 'synopsis' bertipe 'String', 'imageResId' bertipe 'Int', dan 'webtoonUrl' bertipe 'String'. Data class ini berfungsi sebagai model data yang merepresentasikan satu item komik dalam aplikasi.

ComicDataSource.kt

File 'ComicDataSource.kt' berfungsi sebagai sumber data statis untuk daftar komik. Object 'ComicDataSource' memiliki fungsi 'getComicList()' yang menerima parameter 'context' bertipe 'Context' dan mengembalikan list berisi lima objek 'Comic'. Setiap objek 'Comic' diinisialisasi dengan data seperti judul, genre, author, tahun, sinopsis yang diambil menggunakan 'context.getString()', resource gambar menggunakan 'R.drawable', dan URL webtoon masing-masing komik.

ComicViewModel.kt

Class 'ComicViewModel' mewarisi 'ViewModel' dan menerima satu parameter yaitu 'appName' bertipe 'String'. Data list komik dikelola menggunakan 'MutableStateFlow' yang di-expose sebagai 'StateFlow' melalui properti 'comicList', event navigasi ke detail dikelola melalui properti 'selectedComicId', dan event membuka URL dikelola melalui properti 'openUrlEvent'. Fungsi 'loadComics()' memuat data dari 'ComicDataSource.getComicList()' dan menyimpannya ke '_comicList.value', kemudian Timber mencatat log jumlah data yang dimuat dan setiap item komik. Fungsi 'onDetailClicked()' mencatat log judul dan genre komik yang dipilih melalui 'Timber.d()' lalu mengisi '_selectedComicId.value' dengan 'comicId'. Fungsi 'onReadClicked()' mencatat log URL dan judul komik lalu mengisi '_openUrlEvent.value' dengan URL. Fungsi 'onNavigatedToDetail()' mencatat log judul, author, dan tahun komik yang dipilih lalu mereset '_selectedComicId.value' menjadi null.

ComicViewModelFactory.kt

Class 'ComicViewModelFactory' mengimplementasikan 'ViewModelProvider.Factory' dan menerima satu parameter yaitu 'appName' bertipe 'String'. Di dalam fungsi 'create()', dilakukan pengecekan apakah 'modelClass' sesuai dengan 'ComicViewModel::class.java', dan jika sesuai maka instance 'ComicViewModel' dibuat dengan meneruskan 'appName' sebagai argumen menggunakan '@SuppressWarnings("UNCHECKED_CAST")' untuk menghindari peringatan cast yang tidak bisa diverifikasi.

ComicListScreen.kt

File 'ComicListScreen.kt' mendefinisikan tiga fungsi composable yaitu 'ComicListScreen()', 'ComicHighlightCard()', dan 'ComicItem()'. Fungsi 'ComicListScreen()' menerima parameter 'navController' bertipe 'NavController' dan 'viewModel' bertipe 'ComicViewModel'. Data dikumpulkan dari 'viewModel.comicList', 'viewModel.selectedComicId', dan 'viewModel.openUrlEvent' menggunakan 'collectAsState()'. 'LaunchedEffect' pertama memanggil 'viewModel.loadComics()' saat pertama kali composable ditampilkan, 'LaunchedEffect' kedua memantau 'selectedComicId' untuk memicu navigasi ke halaman detail menggunakan 'navController.navigate()', dan 'LaunchedEffect' ketiga memantau 'openUrlEvent' untuk membuka URL menggunakan 'Intent.ACTION_VIEW'. Tampilan utama menggunakan 'LazyColumn' yang berisi 'LazyRow' untuk menampilkan 'ComicHighlightCard' dan daftar 'ComicItem' secara

vertikal. Fungsi 'ComicHighlightCard()' menampilkan card komik berukuran '160dp' x '240dp' menggunakan 'Card' dengan 'RoundedCornerShape' dan modifier 'clickable'. Fungsi 'ComicItem()' menampilkan card komik horizontal menggunakan 'Card' dengan 'RoundedCornerShape' yang berisi gambar cover, informasi komik, serta tombol 'btnRead' dan 'btnDetail'.

ComicDetailScreen.kt

File 'ComicDetailScreen.kt' mendefinisikan fungsi composable 'ComicDetailScreen()' yang menerima parameter 'comicId' bertipe 'Int?' dan 'viewModel' bertipe 'ComicViewModel'. Data list komik dikumpulkan dari 'viewModel.comicList' menggunakan 'collectAsState()'. 'LaunchedEffect' dipanggil untuk memuat data menggunakan 'viewModel.loadComics()' jika 'comicList' masih kosong. Komik yang sesuai dicari menggunakan 'comicList.find' berdasarkan 'comicId'. Jika komik ditemukan, tampilan dirender menggunakan 'Column' dengan modifier 'verticalScroll' yang berisi 'Image' untuk cover komik dengan tinggi '350dp' dan 'ContentScale.Crop', diikuti informasi judul, tahun, genre, author, label sinopsis, dan isi sinopsis komik.

LanguageScreen.kt

File 'LanguageScreen.kt' mendefinisikan fungsi composable 'LanguageScreen()' yang menerima parameter 'navController' bertipe 'NavController'. Context didapatkan menggunakan 'LocalContext.current'. Tampilan menggunakan 'Column' dengan 'verticalArrangement' 'Arrangement.Center' dan 'horizontalAlignment' 'Alignment.CenterHorizontally'. Terdapat dua buah 'Button' yaitu tombol bahasa Inggris yang saat ditekan membuat 'Locale' baru dengan kode '"en"', menerapkannya menggunakan 'Locale.setDefault()' dan 'context.resources.updateConfiguration()', lalu memanggil 'navController.popBackStack()', dan tombol bahasa Indonesia yang melakukan hal serupa dengan kode '"in"'.

Timber Logging

Timber adalah library logging untuk Android yang dikembangkan oleh Jake Wharton sebagai alternatif dari 'Log' bawaan Android. Timber mempermudah proses logging karena tidak membutuhkan tag yang ditulis secara manual di setiap baris pemanggilan log. Inisialisasi Timber dilakukan satu kali di 'MyApp' menggunakan 'Timber.plant(Timber.DebugTree())' sehingga

logging langsung dapat digunakan di seluruh bagian aplikasi menggunakan 'Timber.d()' untuk mencatat debug log.

Pada aplikasi MyComicApp, Timber digunakan untuk mencatat empat jenis event. Pertama, log dicatat saat data komik berhasil dimuat ke dalam list di dalam fungsi 'loadComics()' pada 'ComicViewModel' dengan pesan yang menyertakan nama appName dan jumlah komik yang berhasil dimuat. Kedua, log dicatat saat tombol Detail ditekan melalui fungsi 'onDetailClicked()' dengan pesan yang menyertakan judul dan genre komik yang dipilih. Ketiga, log dicatat saat tombol Read ditekan melalui fungsi 'onReadClicked()' dengan pesan yang menyertakan URL dan judul komik. Keempat, log dicatat saat pengguna berpindah ke halaman detail melalui fungsi 'onNavigatedToDetail()' dengan pesan yang menyertakan judul, author, dan tahun komik yang dipilih.

Debugger

Debugger adalah fitur pada Android Studio yang digunakan untuk memeriksa jalannya program secara langkah demi langkah ketika aplikasi sedang berjalan. Dengan menggunakan Debugger, pengembang dapat menemukan dan memperbaiki bug dengan cara menghentikan eksekusi program pada titik tertentu yang disebut breakpoint, kemudian memeriksa nilai variabel dan alur eksekusi kode pada titik tersebut secara langsung.

Cara menggunakan Debugger di Android Studio adalah dengan terlebih dahulu menentukan breakpoint pada baris kode yang ingin diperiksa dengan cara mengklik di bagian kiri nomor baris kode tersebut hingga muncul titik merah. Setelah itu aplikasi dijalankan menggunakan tombol Debug yang memiliki ikon serangga, bukan tombol Run biasa. Ketika eksekusi program mencapai baris yang dipasang breakpoint, aplikasi akan berhenti secara otomatis dan panel Debug di bagian bawah Android Studio akan menampilkan nilai seluruh variabel yang aktif pada saat itu.

Pada aplikasi MyComicApp, breakpoint dipasang pada baris 'val list = ComicDataSource.getComicList(context)' di dalam fungsi 'loadComics()' pada 'ComicViewModel'. Breakpoint ini relevan karena merupakan titik di mana data komik pertama kali diambil dari sumber data dan akan disimpan ke dalam 'StateFlow', sehingga nilai variabel 'list' yang berisi seluruh data komik dapat diperiksa secara langsung di panel Debug.

Fitur Step Over digunakan untuk melanjutkan eksekusi ke baris berikutnya dalam fungsi yang sama tanpa masuk ke dalam fungsi lain yang dipanggil pada baris tersebut. Pada aplikasi MyComicApp, Step Over digunakan untuk melanjutkan eksekusi dari baris 'val list = ComicDataSource.getComicList(context)' ke baris '_comicList.value = list' berikutnya sehingga dapat dilihat perubahan nilai variabel secara bertahap. Fitur Step Into digunakan untuk masuk ke dalam fungsi yang dipanggil pada baris yang sedang aktif. Pada aplikasi MyComicApp, Step Into digunakan untuk masuk ke dalam fungsi 'getComicList()' pada 'ComicDataSource' sehingga proses pembuatan setiap objek 'Comic' dapat diperiksa satu per satu beserta nilainya. Fitur Step Out digunakan untuk keluar dari fungsi yang sedang dijalankan dan kembali ke fungsi pemanggil. Pada aplikasi MyComicApp, Step Out digunakan untuk keluar dari fungsi 'getComicList()' dan kembali ke konteks fungsi 'loadComics()' pada 'ComicViewModel'.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya.

A. Pembahasan

Application class merupakan sebuah class khusus yang tersedia dalam arsitektur aplikasi Android, dimana class ini menjadi komponen yang pertama kali dibuat ketika sebuah aplikasi dijalankan, bahkan sebelum Activity maupun Fragment apapun sempat terbentuk. Class ini dibuat dengan cara mewarisi atau meng-extend class 'Application' yang sudah disediakan oleh Android, dan memiliki lifecycle yang mengikuti lifecycle aplikasi secara keseluruhan. Artinya, selama aplikasi masih aktif berjalan di perangkat, instance dari Application class ini akan terus ada dan baru akan dihancurkan ketika aplikasi benar-benar ditutup oleh sistem maupun pengguna.

Salah satu fungsi utama dari Application class adalah sebagai tempat yang tepat untuk melakukan inisialisasi berbagai komponen yang bersifat global, yaitu komponen yang hanya perlu diinisialisasi satu kali saja selama siklus hidup aplikasi berlangsung. Sebagai contoh konkret, inisialisasi library logging seperti Timber dilakukan dengan memanggil 'Timber.plant(Timber.DebugTree())' di dalam Application class, sehingga proses logging sudah aktif bahkan sebelum Activity pertama dibuat. Selain itu, Application class juga lazim digunakan sebagai tempat inisialisasi library pihak ketiga lainnya, konfigurasi dependency injection seperti Hilt atau Dagger, serta berbagai pengaturan global yang dibutuhkan oleh aplikasi.

Fungsi lain yang tidak kalah penting adalah Application class dapat berperan sebagai wadah penyimpanan state atau data yang perlu dapat diakses secara global oleh seluruh komponen dalam aplikasi. Karena sifatnya yang singleton, yaitu hanya memiliki satu instance tunggal selama aplikasi berjalan, maka Application class dapat menyimpan referensi ke komponen-komponen tertentu yang sewaktu-waktu dibutuhkan dari berbagai bagian aplikasi tanpa perlu membuat instance baru setiap kali diperlukan. Untuk mendaftarkan Application class agar dikenali oleh sistem Android, class tersebut wajib didaftarkan di dalam file 'AndroidManifest.xml' dengan menambahkan atribut android:name pada 'tag <application>'.
</div>

TAUTAN GIT

Berikut adalah tautan untuk *source code* yang telah dibuat.

<https://github.com/zvyaa/MODUL-MOBILE>